Git & GitHub: Complete Notes

Oct 8, 2026 · @omkar

Git ek distributed version control system hai aur GitHub uska online hosting platform. Ye notes 14 concepts me hain, har concept ke end me interview ka quick check hai.

1. Version Control System (VCS) basics

VCS ek tool hai jo code ki har change ki history save karta hai, taaki "final_v2_real" jaise folders ki zarurat na pade.

Benefits

- **Tracking:** kisne, kab, kya aur kyun badla, sab record hota hai.
- **Collaboration:** kai developers ek project pe kaam karte hain bina ek-dusre ka code overwrite kiye.
- **Rollback:** kuch toot gaya to purane working version pe wapas jao.
- **Backup:** code remote server pe bhi rehta hai.

Centralized vs Distributed

	Centralized (SVN, CVS)	Distributed (Git)
History kahan	Sirf central server pe	Har developer ke paas poori copy
Offline kaam	Nahi, commit ke liye server chahiye	Haan, commit, log, branch sab local
Speed	Network pe depend, slow	Fast, sab local
Server crash	Poori history risk me	Kisi bhi clone se recover

Git ek distributed VCS hai, Linus Torvalds ne 2005 me Linux kernel ke liye banaya.

Interview trap: Git ek tool hai (laptop pe chalta hai), GitHub ek website hai jo Git repos host karti hai (GitLab, Bitbucket bhi aise hi hain). GitHub down ho jaye to bhi local commit, branch, log chalte rehte hain, sirf push/pull rukta hai.

2. Setting up Git

Install: git-scm.com/downloads (Windows pe Git for Windows, jisme Git Bash aata hai). Check: `git --version`.

Git ko batana padta hai tum kaun ho, kyunki har commit pe naam aur email attach hota hai.

```
git config --global user.name "Omkar"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --list
```

- `user.email` wahi rakho jo GitHub account pe hai, warna commits contribution graph me nahi dikhte.
- `init.defaultBranch main`: naye repo ki pehli branch `main` hogi (pehle `master` tha).
- Global config `~/.gitconfig` me store hoti hai.

Config levels

Level	Flag	Scope
System	`--system`	Computer ke saare users
Global	`--global`	Tumhare user ke saare repos
Local	`--local` (default)	Sirf current repo

Priority: Local > Global > System. Kisi ek repo me alag email chahiye to us repo ke andar bina `--global` ke `git config user.email "work@company.com"` chalao.

Optional: `git config --global core.editor "code --wait"` se commit message VS Code me likhne ko milta hai (default Vim hai).

Quick check: name/email set nahi hai aur `git commit` kiya to? Git "Please tell me who you are" error dega aur commit nahi hoga.

3. Basic Git commands

```
git init                  # folder ko repo banata hai (.git folder)
git status                # kaunsi files untracked / modified / staged
git add Main.java         # ek file staging me
git add .                 # saari changes staging me
git commit -m "message"   # staged changes ka snapshot
git commit -am "message"  # tracked files add + commit
git log                   # poori history
git log --oneline         # ek line per commit
```

- `.git` folder me poori history rehti hai. Delete kiya to history khatam.
- `git init` har project me sirf ek baar.
- `git status` safe hai, kuch change nahi karta. Jab confuse ho, chalao.
- `git add .` andhadhund mat chalao, pehle status dekho, warna `.env` jaisi files stage ho sakti hain.
- Commit ek snapshot hai (sirf diff nahi), har commit ka unique hash hota hai.
- Sirf staged changes commit hote hain. `commit -am` naye (untracked) files pe kaam nahi karta.

Poora flow

```
git init
echo "hello" > Test.java
git add Test.java
git commit -m "Add Test.java"
git log --oneline
```

Quick check: `Main.java` edit ki par `git add` nahi kiya, `git commit -m "update"` chalaya to? "Nothing added to commit", kyunki staging khali hai.

4. Git workflow: 3 areas

```
[Working Directory] --(git add)--> [Staging Area] --(git commit)--> [Local
Repository]
```

Area	Kya hai
Working Directory	Actual project files jahan tum edit karte ho

Area	Kya hai
Staging Area (Index)	"Tray" jisme decide hota hai ki agle commit me kya jayega
Local Repository	`.git` ke andar committed snapshots, yahi history hai

Staging kyun? 5 files badli, par 2 login feature ki aur 3 alag fixes ki. Staging se do clean, logical commits ban sakte hain. Trade-off: ek extra step, par control milta hai.

File lifecycle

State	Matlab
Untracked	Nayi file, Git ko abhi pata nahi
Staged	Agle commit ke liye ready (`git add` ke baad)
Committed (Unmodified)	Repo me safe
Modified	Tracked file me edit hua, staged nahi

Flow: Untracked, `add`, Staged, `commit`, Committed, edit karo, Modified, `add`, Staged, aur aise hi chalta hai.

Tricky case: file stage karne ke baad dobara edit kiya to wo do jagah dikhti hai (staged me purana version, modified me naya). Commit me sirf staged version jata hai. Nayi edit ke liye dobara `git add`.

```
git diff           # working dir vs staging (unstaged changes)
git diff --staged  # staging vs last commit (jo commit hone wala hai)
```

Quick check: `A.java` stage ki, phir ek line aur likhi, commit kiya. Nayi line commit me? Nahi, wo abhi Modified state me hai.

5. Working with GitHub (remote repo)

Local repo tumhare laptop pe hai, remote repo server (GitHub) pe. Steps:

1. github.com pe account banao.

2. New repository banao. Agar local repo pehle se hai to README, .gitignore, license add mat karo (khali repo), warna history mismatch error aa sakta hai.
3. Local ko remote se jodo aur push karo:

```
git remote add origin https://github.com/yourusername/your-repo.git
git remote -v
git push -u origin main
```

- `origin` remote URL ka nickname hai (convention).
- `remote add` sirf link banata hai, upload `git push` se hota hai.
- `-u` upstream set karta hai (local `main` ko `origin/main` se link). Sirf pehli baar lagta hai, uske baad bas `git push` / `git pull`.
- Push sirf committed changes bhejta hai.

Roz ka flow

```
git add .
git commit -m "Add feature"
git push
```

Authentication: GitHub password se push allow nahi karta.

Method	Kaise
HTTPS + Personal Access Token (PAT)	Settings, Developer settings, Tokens. Password ki jagah token
SSH key	Key generate karke GitHub me add karo, `git@github.com:user/repo.git` URL

Beginners ke liye PAT simple hai, long-term ke liye SSH better.

Common error: `failed to push some refs` ka matlab aksar remote pe aisi commits hain jo local me nahi (jaise GitHub pe README add kiya tha). Pehle `git pull origin main`, phir push.

6. Clone aur Pull

```
git clone https://github.com/username/repo.git
git clone https://github.com/username/repo.git my-folder
git pull origin main     # upstream set ho to bas: git pull
```

`git clone` ek command me: naya folder banata hai, poori history aur saari branches download karta hai, remote ko `origin` naam se set karta hai, default branch checkout karta hai. Clone ke baad `git init` ya `git remote add` nahi chahiye. Ek repo ke liye sirf ek baar.

`git pull` = `git fetch` + `git merge`.

- `fetch`: remote se naye commits download karta hai, working files ko touch nahi karta.
- `merge`: downloaded code ko current branch me milata hai.

Safe tareeka (pehle dekho, phir milao):

```
git fetch
git log HEAD..origin/main --oneline    # remote pe kya naya hai
git merge origin/main
```

`A..B` syntax: `git log A..B` ka matlab B me wo commits jo A me nahi hain. `HEAD..origin/main` = remote pe jo naye commits hain. Ulta `origin/main..HEAD` = local ke wo commits jo abhi push nahi hue.

	clone	pull
Kab	Pehli baar, repo local me nahi	Repo already local me hai
Kitni baar	Ek baar	Baar-baar
Kya laata hai	Poora repo + history	Sirf naye commits

Uncommitted changes aur remote ke changes same file me hon to pull rukega ya conflict aayega. Kaam shuru karne se pehle `git pull` karna achhi habit hai.

Quick check: `git fetch` ke baad working files badalti hain? Nahi, sirf `origin/main` update hota hai. Files tab badlengi jab merge ya pull karo.

7. Branching aur Merging

Branch ek alag timeline hai jisme tum `main` ko bigade bina naya kaam kar sakte ho. Andar se branch sirf ek commit ki taraf pointer hai, isliye banana instant aur sasta hai. `HEAD` batata hai tum abhi kaunsi branch pe ho.

```
git branch                   # saari branches (* = current)
git branch new-feature       # banao (switch nahi hota)
git checkout new-feature     # switch
git checkout -b new-feature  # banao + switch
git switch new-feature       # modern (Git 2.23+)
git switch -c new-feature    # modern: banao + switch
```

Merge: pehle us branch pe jao jisme merge karna hai. Rule: tum jis branch pe ho, usme dusri branch aati hai.

```
git switch main
git merge new-feature
git branch -d new-feature    # safe delete (unmerged ho to mana)
git branch -D new-feature    # force delete
git push -u origin new-feature
```

1. Fast-forward merge: `main` pe branch banane ke baad naya commit nahi hua. Git bas pointer aage khiska deta hai, merge commit nahi banta, history seedhi rehti hai.

```
A --- B --- C --- D   (main aur feature dono yahan)
```

2. 3-way merge: dono branches pe alag commits hain.

```
          C --- D        (feature)
         /       \
A --- B --- E --- M      (main, M = merge commit)
```

Git 3 commits compare karta hai: common ancestor (B), main ka tip (E), feature ka tip (D). B ko base banake dekhta hai ki dono ne kya badla, phir badlav combine karke ek merge commit (2 parents) banata hai.

Line	Base (B)	main (E)	feature (D)	Git ka decision
1	class App	class MainApp	class App	Sirf main ne badli, main wali lo

Line	Base (B)	main (E)	feature (D)	Git ka decision
2	login()	login()	login()	Kisi ne nahi badli, same
3	signup()	signup()	signup() with email	Sirf feature ne badli, feature wali lo

Dono changes alag lines pe hain, to auto-merge hota hai. Same line badli ho to conflict (next section).

	Fast-forward	3-way
Kab	Sirf ek branch pe naye commits	Dono branches pe naye commits
Merge commit	Nahi	Banta hai
History	Seedhi line	Branch-merge shape

Branch delete karne se commits delete nahi hote, sirf pointer hatta hai.

Quick check: `feature` pe ho aur `git merge main` chalaya, kaun kisme? `main` ka code `feature` me aayega, merge hamesha current branch me hota hai.

8. Merge Conflicts

Conflict tab hota hai jab dono branches ne same file ki same line alag tareeke se badli ho (ya ek ne file delete ki aur dusri ne edit). Git decide nahi kar sakta, to ruk kar tumse manually resolve karwata hai. Ye error nahi, collaboration me normal cheez hai.

```
Auto-merging App.java
CONFLICT (content): Merge conflict in App.java
Automatic merge failed; fix conflicts and then commit the result.
```

File me Git markers daalta hai:

```
<<<<<<< HEAD
class MainApp
=======
class FeatureApp
>>>>>>> feature
```

Marker	Matlab
`<<<<<<< HEAD`	Yahan se current branch (main) ka code
`=======`	Dono versions ka separator
`>>>>>>> feature`	Yahan incoming branch (feature) ka code khatam

Resolve karne ke 4 steps

1. Conflicted file kholo (`git status` batata hai, "both modified").
2. Decide karo: sirf apna, sirf incoming, ya dono ka mix.
3. Saare markers (`<<<<<<<` , `=======` , `>>>>>>>`) hatao, sirf clean code bachao.
4. Stage aur commit:

```
git add App.java
git commit
```

Ghabra gaye to `git merge --abort` merge se pehle wali state pe wapas le jata hai.

Tools: VS Code (Accept Current / Incoming / Both buttons), IntelliJ (Git, Resolve Conflicts, 3-pane editor).

Conflict kam kaise karein: branch chhoti aur short-lived rakho, kaam se pehle `git pull` , beech-beech me `git merge main` , team me baat karo ki kaun kaunsi file pe hai.

Common galti: markers hatana bhool kar commit kar dena. Commit se pehle `<<<<<<<` search kar lo.

Quick check: resolve karke `git add` kiya par `git commit` nahi, merge complete hua? Nahi, `git status` abhi "still merging" dikhayega.

9. .gitignore

`.gitignore` ek text file hai jo Git ko batati hai kaunsi files track nahi karni. Ye project root me banti hai aur khud commit hoti hai, taaki poori team same rules use kare.

Category	Examples
Build output	`target/` , `build/` , `*.class` , `node_modules/`
IDE settings	`.idea/` , `.vscode/` , `*.iml`

Category	Examples
Secrets	`.env`, `application-local.properties`
Logs	`*.log`
OS files	`.DS_Store`, `Thumbs.db`

Sabse important: secrets (DB password, JWT secret, API keys) kabhi GitHub pe nahi jane chahiye. Public repo me bots minutes me scrape kar lete hain.

Java / Spring Boot example

```
# Build
target/
*.class
*.jar

# IDE
.idea/
.vscode/
*.iml

# Logs
*.log

# Secrets
.env
application-local.properties

# OS
.DS_Store
Thumbs.db
```

Ready-made file: gitignore.io pe "Java, Maven, IntelliJ" select karo.

Pattern syntax

```
*.log            # saari .log files
target/          # folder (aur uske andar sab)
/config.txt      # sirf root wali
!important.log   # exception, ye ignore mat karo
docs/*.pdf       # docs folder ki saari pdf
# comment
```

Sabse badi gotcha: `.gitignore` sirf untracked files pe kaam karta hai. Jo file pehle se commit ho chuki hai usse Git track karta rehta hai. Fix (disk se delete nahi hoti):

```
git rm --cached application-local.properties
git rm -r --cached target/
git commit -m "Stop tracking local files"
```

Secret push ho gaya to: sirf file delete karna kaafi nahi, wo purani commits ki history me rehta hai. Pehla aur zaruri step: us password/key ko turant rotate karo. History se hatana (`git filter-repo`) advanced kaam hai.

Check karne ke liye: `git check-ignore -v .env` (kaunsa rule file ko ignore kar raha hai).

Quick check: `.env` pehle commit ki, phir `.gitignore` me daali, Git ignore karega? Nahi, pehle `git rm --cached .env` karna padega.

10. Undoing changes

Undo ka tareeka is pe depend karta hai ki change kis stage me hai.

Situation	Modern command	Purana command
Working dir ke edits discard	`git restore <file>`	`git checkout -- <file>`
Staged file unstage	`git restore --staged <file>`	`git reset HEAD <file>`
Last commit fix (unpushed)	`git commit --amend`	same
Pushed commit safely undo	`git revert <hash>`	same

1. Discard: `git restore Main.java`. Changes permanently gayab ho jate hain (kabhi commit hi nahi hue the). Pehle `git diff` dekh lo.

2. Unstage: `git restore --staged Main.java`. File Modified state me wapas aati hai, edits safe rehte hain.

3. Amend (last commit me galti): commit ke turant baad typo ya bhuli hui file ke liye.

```
git commit --amend -m "Add login API"    # sirf message badlo
git add JwtUtil.java
git commit --amend --no-edit             # bhuli file jodo, message same
git commit --amend                       # editor me message edit
```

Amend purane commit ko replace karta hai, naya hash milta hai (history rewrite). Jo commit push ho chuka hai usse amend mat karo, teammates ke repo se mismatch hoga.

4. Revert: `git revert a1b2c3d` ek naya commit banata hai jo purane commit ka ulta karta hai. History intact rehti hai, isliye shared/pushed branch pe yahi use karo.

```
A --- B --- C --- C'      (C' = C ka ulta)
```

5. Reset: branch pointer ko peeche khiskata hai. Sirf local, unpushed commits ke liye.

```
git reset --soft HEAD~1     # commit hatao, changes STAGED
git reset --mixed HEAD~1    # commit hatao, changes UNSTAGED (default)
git reset --hard HEAD~1     # commit hatao, changes DELETE
```

Mode	Commit	Staging	Working dir
`--soft`	Hat gaya	Changes bache	Changes bache
`--mixed`	Hat gaya	Khali	Changes bache
`--hard`	Hat gaya	Khali	Changes delete

Revert vs Reset

	revert	reset
Kya karta hai	Ulta commit add karta hai	History se commit hata deta hai
History	Safe	Rewrite hoti hai
Shared/pushed branch	Haan, safe	Nahi, dangerous

Trade-off: revert se history me extra commits dikhte hain, reset se clean rehti hai par teammates ke repo se mismatch ka risk hai.

Safety net: `git reflog` har jagah ki list deta hai jahan HEAD gaya. Wahan se hash dhundh ke `git reset --hard <hash>`. Ye sirf committed cheezon ke liye kaam karta hai.

Quick check: galat commit `main` pe push ho gaya aur teammates ne pull kar liya, reset ya revert? Revert.

11. Fork, Pull Request, Issues

Fork kisi aur ke repo ki apne GitHub account me copy hai. Open-source repo pe write access nahi hota (seedha push pe "permission denied"), isliye pehle fork karte ho.

	Fork	Clone
Kahan	GitHub pe (server pe copy)	Tumhare laptop pe
Kaise	GitHub ka button	`git clone` command

Dono saath use hote hain: pehle fork, phir apne fork ko clone.

Pull Request (PR) = "meri branch ke changes original repo me merge kar do" ki request. Isme diff, discussion, maintainer ka code review aur automated checks (CI) hote hain. Approve hone pe maintainer merge karta hai.

Contribution flow

```
# 1. GitHub pe Fork button, phir:
git clone https://github.com/YOUR-USERNAME/project.git
cd project
git switch -c fix-typo
# ... changes ...
git add .
git commit -m "Fix typo in README"
git push -u origin fix-typo
# 2. GitHub pe "Compare & pull request" dabao
```

Review me changes maange jayein to usi branch me commit karke push karo, PR automatically update hota hai.

Fork ko sync rakhna

```
git remote add upstream https://github.com/ORIGINAL-OWNER/project.git
git fetch upstream
git switch main
git merge upstream/main
```

`origin` = tumhara fork, `upstream` = original repo (standard naming).

Issues: bug report, feature request, task tracking aur sawal ki jagah. Labels (`bug` , `enhancement` , `good first issue`), assignee aur comments hote hain. Pehle open-source contribution ke liye `good first issue` label dhundo. PR description me `Fixes #12` likhne se merge hote hi issue #12 close ho jata hai.

Fork vs Branch: apni team ke repo me (write access hai) fork nahi karte, seedha branch banake PR karte ho. Fork tab jab original me write access nahi.

Quick check: fork ke `main` pe push kiya, original me change aaya? Nahi, original me change tab aayega jab PR banao aur maintainer merge kare.

12. Advanced Git: Rebase, Stash, Tag, Cherry-pick

Rebase

Rebase tumhari branch ke commits ko target branch ke latest commit ke upar dobara laga deta hai.

```
Pehle:   A --- B --- E --- F          (main)
                \
                 C --- D              (feature)

Baad me: A --- B --- E --- F --- C' --- D'   (feature)
```

```
git switch feature
git rebase main
# conflict aaye: resolve, git add <file>, git rebase --continue
# ghabra gaye:   git rebase --abort
```

C′ aur D′ naye commits hain (naye hash), yani rebase bhi history rewrite hai. Merge commit nahi banta, history seedhi line me rehti hai.

	Merge	Rebase
History	Branch-merge shape, merge commit	Seedhi, clean line
Commits	Jaise the waise	Naye hash ke saath rewrite
Shared branch pe	Safe	Dangerous

Golden rule: jo branch push ho chuki hai aur dusre use kar rahe hain, usse rebase mat karo. Safe use: apni local, unpushed feature branch ko `main` ke upar rebase karna.

Stash

Adhoora kaam temporarily side me rakh do, working directory clean ho jati hai. Use case: urgent bug ke liye branch switch karni hai.

```
git stash                    # kaam side me
git stash push -m "login wip"  # naam ke saath
git stash -u                 # untracked files bhi
git stash list
git stash apply              # wapas laao (stash bacha rehta hai)
git stash pop                # wapas laao + list se hatao
git stash drop               # ek stash delete
```

```
git stash
git switch main     # urgent bug fix karo
# ... fix, commit, push ...
git switch feature
git stash pop
```

Tag

Kisi specific commit ko permanent label (usually release ke liye). Branch naye commits ke saath aage move hota hai, tag fixed rehta hai.

```
git tag v1.0                       # lightweight
git tag -a v1.0 -m "First release" # annotated (releases ke liye recommended)
git tag v0.9 a1b2c3d               # purane commit pe
git tag                            # list
git push origin v1.0               # ek tag push
git push origin --tags             # saare tags
git tag -d v1.0                    # local delete
```

`git push` normally tags nahi bhejta, alag se push karna padta hai. Naming: semantic versioning `vMAJOR.MINOR.PATCH` (MAJOR = breaking change, MINOR = naya feature, PATCH = bug fix). GitHub pe tags se Releases banti hain.

Cherry-pick

Dusri branch ka sirf ek specific commit apni current branch pe copy karna.

```
git switch main
git cherry-pick c3d4e5f
git cherry-pick c3d4e5f a1b2c3d     # kai commits
git cherry-pick A..B                # range
# conflict: resolve, git add, git cherry-pick --continue
git cherry-pick --abort
```

C ki copy (C′) naye hash ke saath aati hai, original C `feature` me waisa hi rehta hai. Use case: hotfix ko `main` aur `release` dono me chahiye, ya galat branch pe commit ho gaya. Ye exception ke liye hai, roz ke kaam me merge/rebase use hota hai, kyunki same change do jagah alag hash se baad me duplicate conflicts de sakta hai.

Quick check: rebase ke baad C aur D ke hash wahi rehte hain? Nahi, naye hash milte hain. Tag aur branch ka fark? Branch aage move hota hai, tag ek commit pe fixed rehta hai.

13. Best practices

Ye commands nahi, habits hain. Interview me "Git best practices bataiye" aksar poochha jata hai.

1. Clear commit messages. Message batata hai kya aur kyun badla.

```
Bad:   "fix", "changes", "asdf"
Good:  "Fix NPE in JwtFilter when token is missing"
Good:  "Add rate limiting to login API"
```

- Imperative mood me likho ("Add", "Fix", "Remove"), socho "This commit will... Add login API".
- Pehli line ~50 characters tak. Zyada detail ho to body me: `git commit -m "Fix NPE in JwtFilter" -m "Token was null when header was absent. Added null check, return 401."`
- Popular style: Conventional Commits (`feat: add login`, `fix: handle null token`, `docs: update README`).

2. Chhote, logical commits. Ek commit = ek kaam. Review, bug dhundhna (`git bisect`) aur revert, teeno easy ho jate hain.

3. Branches use karo. `main` hamesha working rakho. Naming: `feature/login`, `feature/jwt-refresh`, `fix/null-token`. Branch short-lived rakho, warna conflicts badhte hain.

4. Kaam se pehle pull.

```
git switch main
git pull
git switch -c feature/new-thing
```

5. Push se pehle check karo.

```
git status
git diff --staged
git log origin/main..HEAD --oneline    # kya push hone wala hai
```

6. Secrets kabhi commit mat karo. `.env`, passwords, API keys, JWT secrets `.gitignore` me. Push ho gaya to key rotate karo.

7. Shared history rewrite mat karo. `amend`, `reset`, `rebase` sirf local, unpushed commits pe. Pushed pe `revert`.

8. Frequently commit aur push karo. Laptop kharab hua to sab gaya. Regular push = backup bhi aur team ko progress bhi dikhta hai.

9. PR me achhi description. Kya badla, kyun badla, kaise test kiya. `Fixes #12` se issue link karo.

Quick check: ek hi commit me login feature, README typo aur pom.xml update daal diya to? Review mushkil, aur sirf login revert karna ho to baaki changes bhi ud jayenge. Teen alag commits hone chahiye the.

14. Tools and resources, plus placement cheat sheet

Graphical interfaces: GitHub Desktop (beginners), GitKraken, Sourcetree. IntelliJ me built-in Git UI hai, Java kaam ke liye handy.

History visualize karna:

```
git log --graph --oneline --all
```

Learning platforms: Learn Git Branching (interactive, sabse best practice ke liye), Git official documentation, GitHub Skills (purana naam GitHub Learning Lab).

Placement ke liye 6 high-ROI cheezein

1. Git vs GitHub ka difference
2. 3 areas: working directory, staging, repository
3. `merge` vs `rebase`
4. `reset` vs `revert`
5. `fetch` vs `pull`
6. Merge conflict kaise resolve karte ho

Command cheat sheet

Kaam	Command
Repo banao	`git init`
Status	`git status`
Stage	`git add .`
Commit	`git commit -m "msg"`
History	`git log --oneline`
Remote jodo	`git remote add origin <url>`
Push (pehli baar)	`git push -u origin main`
Clone	`git clone <url>`
Naya aur switch	`git switch -c <branch>`
Merge	`git merge <branch>`
Edit discard	`git restore <file>`
Unstage	`git restore --staged <file>`
Last commit fix (unpushed)	`git commit --amend`
Pushed commit undo	`git revert <hash>`
Adhoora kaam side me	`git stash` / `git stash pop`
Release label	`git tag -a v1.0 -m "msg"`
Ek commit copy	`git cherry-pick <hash>`